

Phishing detection model

Julian Calvo. AAI6850, Module 15 capstone. August 2026.

This is the model that gets sold on the NexusAI marketplace. Its evaluation output is what the oracle publishes, and the marketplace refuses to sell it if the published accuracy drops below the floor the provider declared.

1. Problem

Binary classification of email as phishing or benign, from the subject, body, sender headers and any URLs in the body.

The costs are asymmetric. A false negative lets a phishing email into an inbox. A false positive quarantines legitimate business mail, and enough of those and the security team turns the filter off. I weighted a false negative at 4x a false positive and required precision of at least 0.90 at whatever operating point I picked.

2. Data

The corpus is synthetic. The machine I built this on can't reach the public phishing datasets, so `ml/data/build_dataset.py` generates records from a seeded RNG (seed 6850) using 114 templates. Loaders for Nazario, SpamAssassin, PhishTank and the UCI phishing dataset are written and get tried first. They have never run against real files here.

	Records	Phishing	Benign	Groups
Corpus	8,000	3,356	4,644	1,574
Holdout	1,200	481	719	805

The holdout includes 300 records from templates that appear nowhere in the corpus. 1.5% label noise is injected deliberately, so a perfect score is not achievable and anything close to one would be a bug.

Leakage control

Templates produce near-duplicate messages. A random train/test split would put siblings of a test message in training and report a score that means nothing. So the split is `StratifiedGroupKFold` on `group_id`, where a group is a near-duplicate cluster.

Split sizes: train 4,797, val 1,603, test 1,600. Group overlap between splits is zero, checked and asserted.

I measured what the grouping is worth by running the identical pipeline both ways:

Split	Test F1
Random	0.9411
Grouped	0.9358

That probe uses a fixed logistic regression pipeline rather than the selected ensemble, so those numbers aren't comparable to the 0.960 in section 6.

So a random split would have overstated F1 by about 0.005. Small here, because the synthesiser spreads templates fairly evenly. On real corpora with heavy campaign duplication it would be much larger, which is why the control stays in.

3. Features

Three families, all built in `ml/src/features.py`.

A. Text. TF-IDF over subject and body. Word 1-2 grams, plus `char_wb` 3-5 grams. The character n-grams are there because deliberate misspellings like "account" and "verify", and homoglyph domains, shatter into unseen tokens under word tokenisation but stay recognisable at character level.

B. URL and lexical, 30 features. Count of URLs, HTTPS ratio, maximum and mean URL entropy, longest URL, risky TLDs, shortener hosts, brand lookalike distance, subject and body length, punctuation counts.

C. Header and authentication, 22 features. SPF, DKIM and DMARC results, sender domain shape (digits, hyphens, length, depth), freemail sender, reply-to present, reply-to mismatch, reply-to freemail, display-name brand mismatch, received hop count, role account.

4. Model ladder

Each step tests one idea rather than sweeping hyperparameters. Validation F1 at threshold 0.5:

Model	What changed	Val F1
M1	Baseline. Word TF-IDF + logistic regression	0.7516
M2	Add char n-grams	0.7830
M3	Drop text entirely, URL + header features only, HistGradientBoosting	0.9705
M4	All three families, calibrated LinearSVC	0.9359
M5	All three families, logistic regression, grouped GridSearchCV	0.9463
M6	Soft vote over M3 and M5	0.9682

The interesting result is M3. Throwing away the text entirely and using only engineered URL and header features beats every text model by a wide margin. Most of the signal in this corpus lives in who sent it and where the links point, not in what it says. That matches the older literature (PILFER made the same argument) and it's the opposite of what I expected going in.

M4 doing worse than M3 is the text block dragging down a model that already had the useful features.

M6 is selected, on validation PR-AUC (0.9753). M3 and M5 fail differently: M3 can't read the text at all, M5 can't model feature interactions like "SPF fails AND reply-to mismatches". Averaging their probabilities trims both error modes.

5. Operating point

Selected threshold 0.2723, chosen to minimise expected cost (FN weighted 4x) subject to precision staying above 0.90.

Threshold	Precision	Recall	F1	Cost
0.272 (selected)	0.9715	0.9657	0.9686	111
0.386 (F1-optimal)	0.9831	0.9553	0.9690	131
0.500 (default)	0.9846	0.9523	0.9682	138

F1 hardly moves across that range but cost moves 25%, so I picked the threshold on cost. F1 wouldn't have distinguished them.

6. Test results

Held out until the model and threshold were both frozen. Evaluated once.

Metric	At 0.272	At 0.5
Accuracy	0.9663	0.9719
Precision	0.9572	0.9816
Recall	0.9629	0.9510
F1	0.9600	0.9660
ROC-AUC	0.9813	0.9813
PR-AUC	0.9740	0.9740

Test F1 0.960 against validation 0.969. A small drop is what you want to see. A test score above validation would suggest the split leaked.

Plots are in `ml/reports/`: ROC and PR curves, confusion matrix, feature importance.

7. Error analysis

29 false positives and 25 false negatives on the test set. Of those, 21 FPs and 9 FNs are genuine model failures. The rest land on records whose label the noise process deliberately flipped, so the model was right and the label was wrong.

Full worked examples are in `ml/reports/error_examples.md`.

False negatives. 12 of the 25 pass all three authentication checks. That's the pattern: a phishing message sent from a genuinely authenticated domain defeats the strongest feature family the model has. One has no URLs at all, which removes the second family too, leaving only the text the model has learned to under-weight.

False positives. Almost all are legitimate mail with no URLs. With no links to score, the URL family contributes only zero-valued features, and several of those zeros push toward phishing (`https_ratio = 0` is the largest single contributor on most of them). The model treats "no links" as suspicious because in this corpus phishing messages more often have no scoreable links.

What I'd do about it. Add an explicit "no URLs present" indicator so the model can distinguish "no links" from "bad links", rather than encoding both as zero. And weight the header family less when authentication passes, since a passing SPF says less than it used to now that attackers can get one.

8. Feature importance

Top drivers toward phishing, from the dense model:

sender_domain_digits, sender_is_freemail, sender_domain_risky_tld, display_brand_mismatch, reply_to_freemail, any_risky_tld, url_max_entropy.

Top drivers toward benign: display_has_brand, brand_lookalike_min_dist, https_ratio, any_brand_lookalike, reply_to_present.

Permutation importance on M3 agrees: sender_is_freemail, sender_domain_digits, reply_to_freemail, received_hops, brand_lookalike_min_dist.

The text features that surface are mostly synthesiser artifacts. time sensitive, kind regards and dropbox are template phrases rather than phishing signal.

9. Ethics

False positives hurt. Quarantined business mail costs real time and, in the wrong context (a legal deadline, a customer escalation), real money. Worse, a noisy filter gets ignored, which makes the false negatives worse too. That's why precision has a floor and isn't just traded off against recall.

False negatives hurt more, but differently. They're what the model exists to prevent, and they're why the cost model weights them 4x.

Privacy. The model reads email content, which is about as sensitive as data gets. In deployment it should run inside the buyer's boundary, not as a service that sees their mail. Nothing in this pipeline ships email content anywhere, and nothing about a buyer or their mail goes on chain. The oracle publishes an accuracy number and a manifest hash, nothing else.

Evasion. An attacker who knows the features can defeat them: get a real domain with valid SPF, avoid digits and hyphens, use plausible display names. The header family is the strongest and also the most gameable. Anyone deploying this should assume it degrades under adversarial pressure and re-evaluate on a schedule. The oracle's staleness window forces that anyway.

Bias. The corpus is English-only, from a synthesiser I wrote, so it inherits my assumptions about what phishing looks like. Non-English mail, unusual business conventions, and legitimate senders on freemail domains are all likely to be treated unfairly. Freemail is a top phishing feature, and small businesses legitimately send from freemail addresses.

10. Limitations

1. The corpus is synthetic. This is the big one.
2. English only.
3. No attachment analysis, no HTML structure, no image content.
4. No temporal split. Real phishing evolves and a chronological holdout would be a harder and more honest test.
5. The authentication features assume headers you can trust, which in practice means you're inside the receiving infrastructure.

6. The threshold was picked on one validation split, not bootstrapped, so there is no confidence interval on it.

11. Future work

Re-run on Nazario, SpamAssassin and PhishTank. Add a temporal split. Try a transformer on the text block and see whether it beats the dense features (I'd guess it helps most on the cases where headers are clean). Add the no-URL indicator from section 7. Bootstrap the threshold. Publish the holdout manifest so a third party can reproduce the oracle score, which the payload already carries the hash for.

12. References

1. Zhang, Hong and Cranor, CANTINA: a content-based approach to detecting phishing web sites, WWW 2007
2. Xiang et al., CANTINA+: a feature-rich machine learning framework for detecting phishing web sites, ACM TISSEC 2011
3. Fette, Sadeh and Tomasic, Learning to detect phishing emails, WWW 2007 (PILFER)
4. Ma et al., Beyond blacklists: learning to detect malicious web sites from suspicious URLs, KDD 2009
5. Le, Markopoulou and Faloutsos, PhishDef: URL names say it all, INFOCOM 2011
6. Pedregosa et al., scikit-learn: machine learning in Python, JMLR 2011